

Chapter 05 — Ingestion: GitHub & Parser Services

Component type: Technical — how raw repo code becomes the pipeline's input. **Source:**

`backend/src/services/github.service.ts` , `backend/src/services/parser.service.ts`

1. GitHub Service — fetching the repo

Uses **Octokit** (GitHub REST). Responsibilities:

1.1 `parseRepoUrl(url) → { owner, repo }`

Regex `github\.com\/([^\/]*)\/([^\/]*)` , strips trailing `.git / /` . Throws on missing/invalid URL.

1.2 `getRepoTree(meta) → TreeNode[]`

`octokit.git.getTree({ tree_sha: "HEAD", recursive: "1" })` — the full default-branch tree in one call. Warns if GitHub truncated it (very large repo). Filters out skipped directories immediately.

Skipped dirs: `node_modules`, `dist`, `build`, `.git`, `coverage`, `.next`, `out`, `__pycache__`, `.vscode`, `vendor` .

1.3 `detectEntryCandidates(tree) → string[]`

Prefers **root-level** entry files; falls back to any-depth matches. **Entry filenames:** `server/index/main/app.{js,ts}` , `main.py` , `app.py` , `Main.java` .

1.4 `getSourceFiles(meta, tree) → FileContent[]` — the batched fetch

```
filter blobs that are source files (.js .ts .jsx .tsx .py .java)
→ cap at MAX_FILES = 60
→ fetch in batches of BATCH_SIZE = 5 (Promise.all per batch)
→ sleep RATE_DELAY_MS = 100ms between batches (rate-limit friendly)
→ skip individual files that error (don't fail the whole batch)
```

Each file is fetched via `getFileContent` (base64 → utf-8) and returned as `{ path, content }` .

1.5 Tuning knobs (currently hardcoded)

Const	Value	Meaning
<code>MAX_FILES</code>	60	max source files fetched
<code>BATCH_SIZE</code>	5	parallel requests per batch

Const	Value	Meaning
RATE_DELAY_MS	100	pause between batches

plan.md proposes moving these to config.

2. Parser Service — code → Internal Representation (IR)

parseRepo(files) turns FileContent[] into ParsedRepo :

```
ParsedRepo = {
  files: ParsedFile[],      // per file: { filePath, language, imports[], exports[], functions[], lin
  sourcePaths: string[],    // all filePaths
}
```

It is **regex-based**, not a **real AST** — fast and dependency-free, but approximate. Language is chosen by extension (Lang_Map).

2.1 JS/TS (parseJSTS)

- **imports:** static `import ... from "x" / import "x", require("x")`, and `export ... from "x"` (re-exports).
- **exports:** `export [default] [async] function|class|const|let|var NAME .`
- **functions:** top-level `function / class` declarations + top-level arrow consts (`const NAME = (...) => .`).

2.2 Python (parsePython)

- **imports:** `import x` and `from x import ... .`
- **functions:** module-level `def / class .`
- **exports:** functions/classes whose name doesn't start with `_` (public convention).

2.3 Java (parseJava)

- **imports:** `import a.b.C; .`
- **exports:** `public class|interface|enum NAME .`
- **functions:** public classes + public methods `public [static] [final] Type name( .`

2.4 Unknown extensions

Return empty `imports/exports/functions` — the file still appears in `sourcePaths` with `language: "unknown"` .

3. How the two feed the pipeline

repoUrl

```
→ parseRepoUrl → getRepoTree → getSourceFiles      (GitHub service)
→ parseRepo(rawFiles) → ParsedRepo                  (parser service)
→ detectEntryCandidates(tree) + resolveEntryContents (M2 enrichment)
→ runAnalysisGraph(parsedRepo, entryCandidates, entryContents)
```

4. Known gaps / improvement ideas

- **Regex parser** misses: dynamic `import()`, multi-line imports, decorators, TS type-only imports, non-top-level functions. Real trade-off vs. bringing in a parser like `@babel/parser` or `tree-sitter`.
- No alias/path resolution — `@/x` imports won't map to a file (hurts M3 accuracy).
- 60-file / 40-file caps mean big repos are partially analyzed silently.
- No caching of GitHub file contents across analyses of the same repo.
- Only 6 extensions supported (no Go, Rust, C#, Ruby, ...).